

Since the dawn of civilization, deception has plagued the humans of the Milky Way Galaxy.

But a new enemy has formed from the depths of darkness, the Deepfake Clones!

All throughout the digital world, false media propagates through the internet and into our senses with their ultimate weapon, the GENERATIVE ADVERSARIAL NETWORKS, and armored with enough memory, storage and compute they aim to control consciousness!!

But there stands a light in the abyss, a rebel army wielding specialized CONVOLUTIONAL NEURAL NETWORKS to detect and destroy the Empire's weapons of evil!

Thus, the war has begun, in hopes to restore truth and order to the Galaxy.....



DEEPPFAKE VIDEO/AUDIO ANALYSIS

Multi-Modal Deepfake Detection using Audio-Visual Incoherence

Steve Elengical

Ali Lalani

Weihao Huang

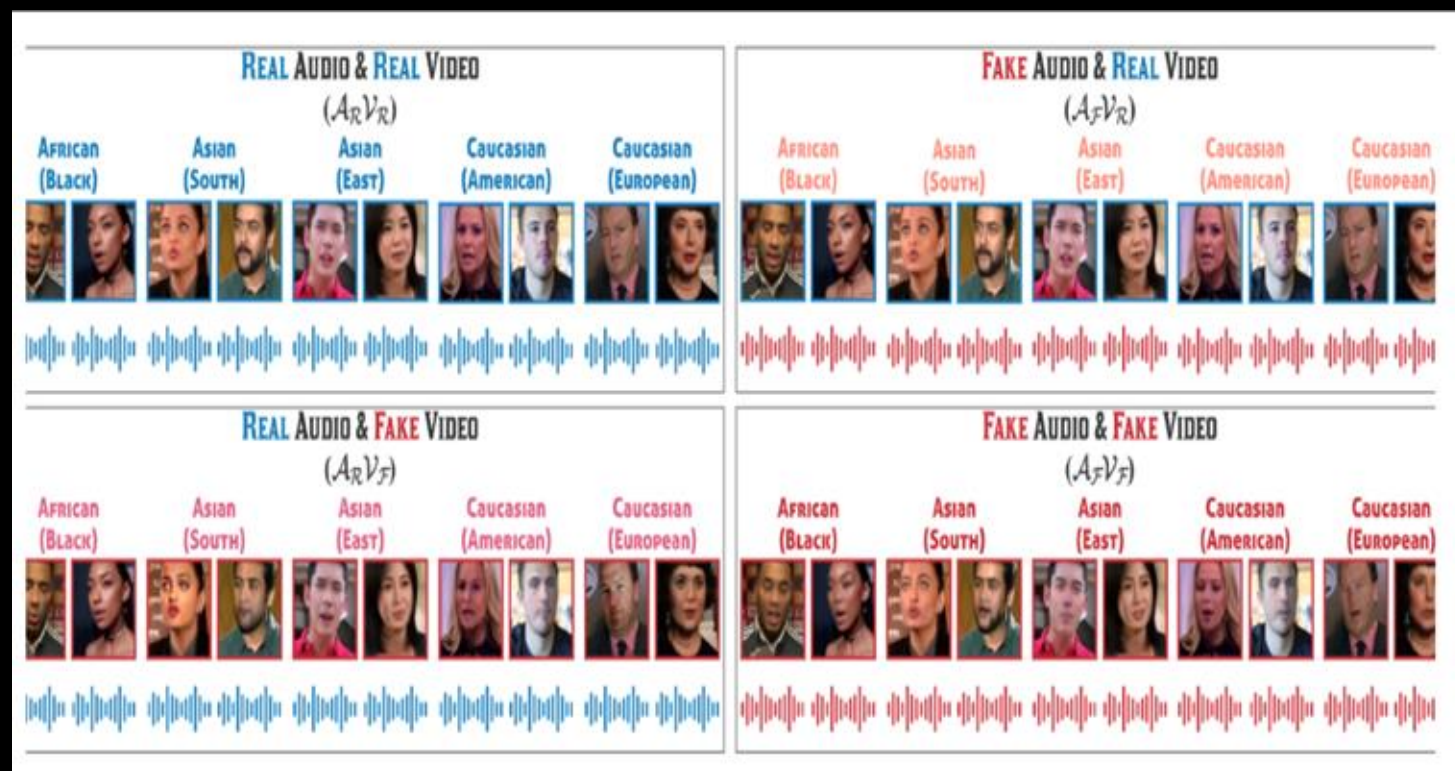
Table of Contents

- Problem Overview
- Dataset & Deepfake Generation
- Data Processing (Audio & Video)
- Model Architecture
- Data Engineering
- Results & Evaluation
- Discussion

Our Dataset – Fake AV Celeb

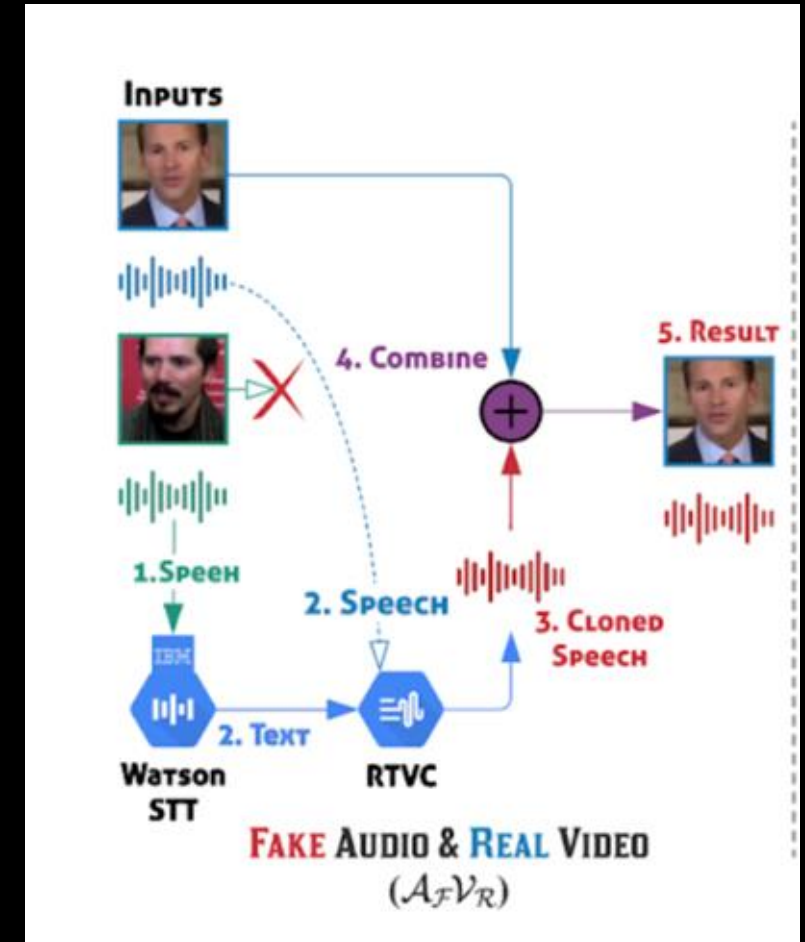
21,544 files

- Primary dataset: **FakeAVCeleb v1.2**
- Total samples: **21,544 audio-video pairs**
- Contains both **real and manipulated (deepfake) content**
- Includes multiple **ethnicities and identities**
- Covers combinations of:
 - a. Real Audio + Real Video
 - b. Fake Audio + Real Video
 - c. Real Audio + Fake Video
 - d. Fake Audio + Fake Video



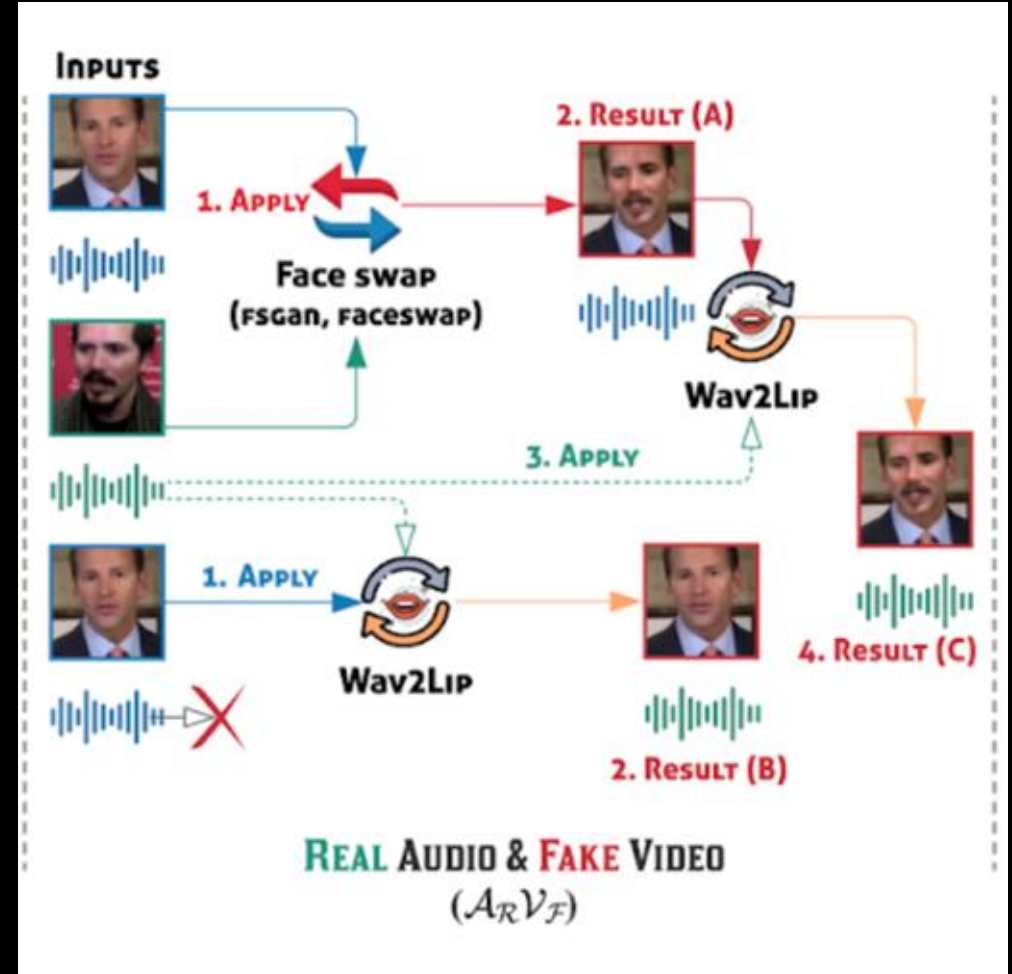
Dataset – Voice Swapping

- Focus: To detect **Audio Manipulation (Voice Cloning)**
- Speech from one person is **converted to another speaker's voice**
- Video remains **unchanged (real)**
- Creates **Audio-Visual Mismatch**
- Helps model detect **inconsistency between voice and face**



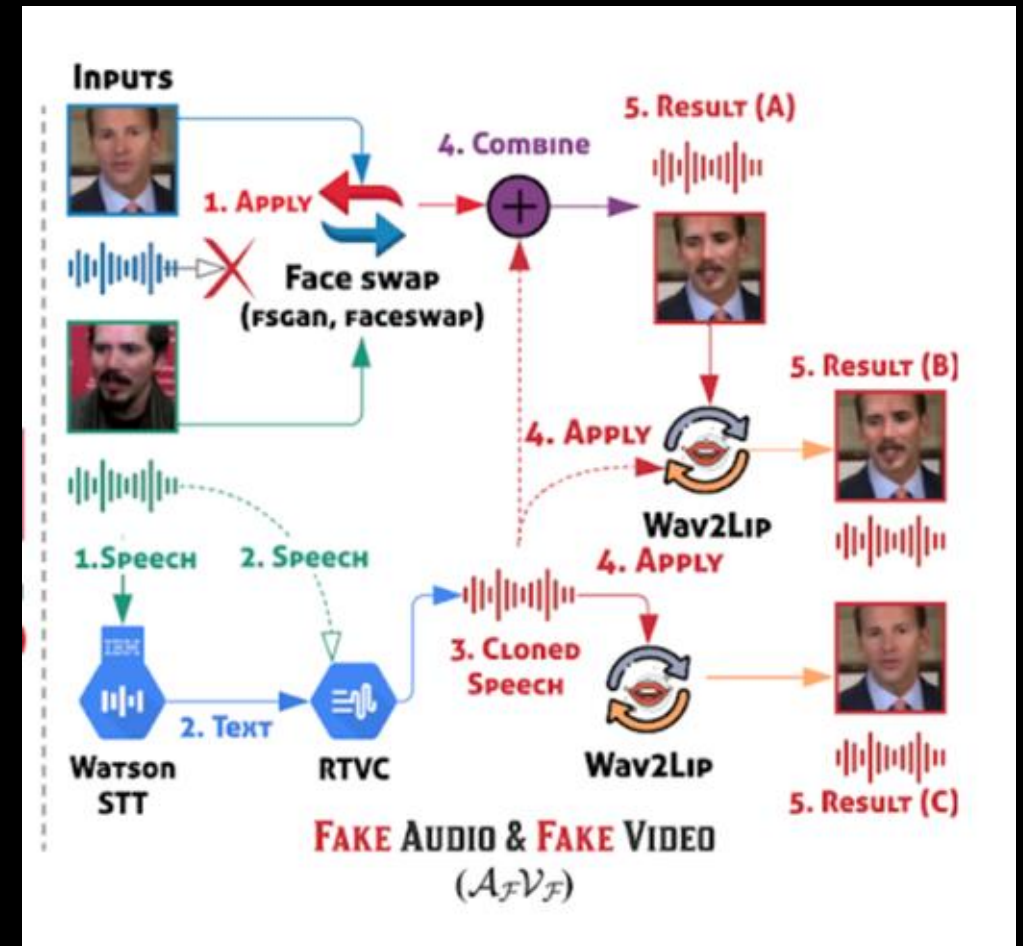
Dataset – FSGAN face swapping

- Focus: **Visual Manipulation (Face Swapping)**
- Face of one person is **replaced with another identity**
- Audio remains **unchanged (real)**
- Uses techniques like **FSGAN** and **Wav2Lip**
- Creates **Real Audio + Fake Video scenarios**
- Helps model detect **visual inconsistencies**



Dataset – Combined Audio/Visual

- We trained our model on ALL possible manipulation scenarios
- Combined **multiple manipulation types** into one dataset
- Includes:
 - Fake Audio + Real Video
 - Real Audio + Fake Video
 - Fake Audio + Fake Video
- Created a **comprehensive training environment**
- Enabled model to learn **cross-modal inconsistencies**
- Improved robustness against **real-world deepfakes**

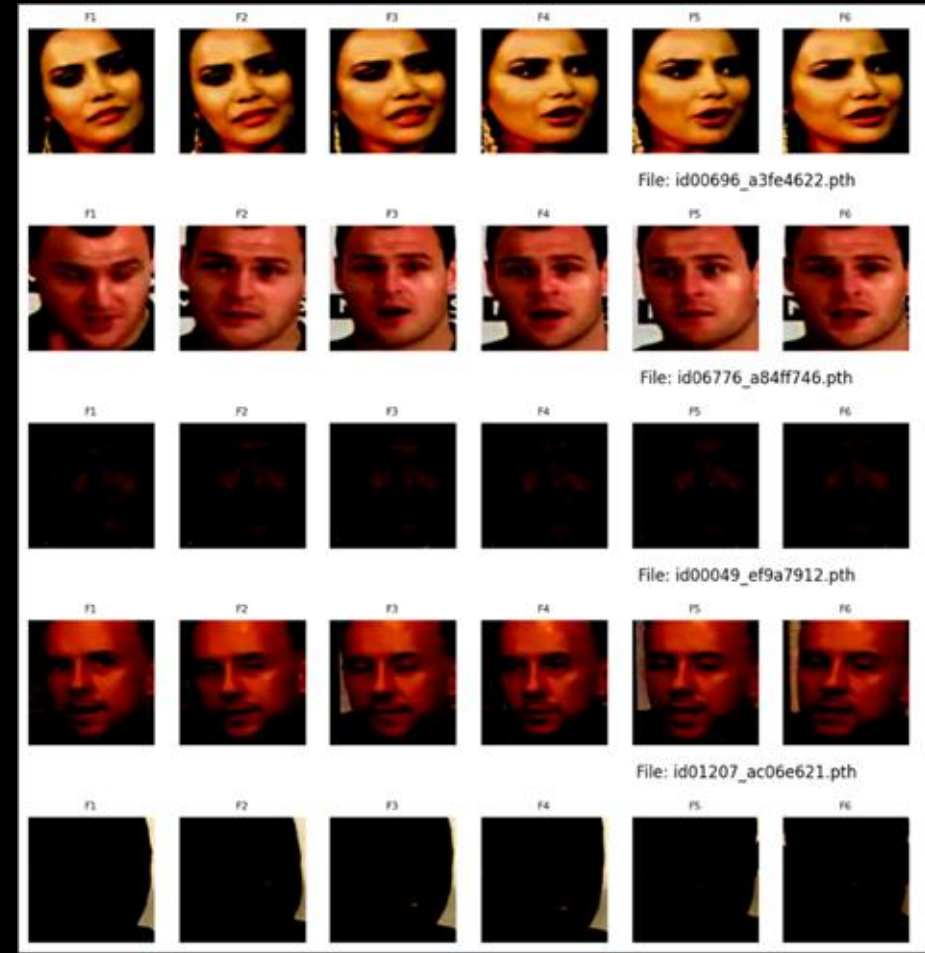




Processing Video & Audio Data

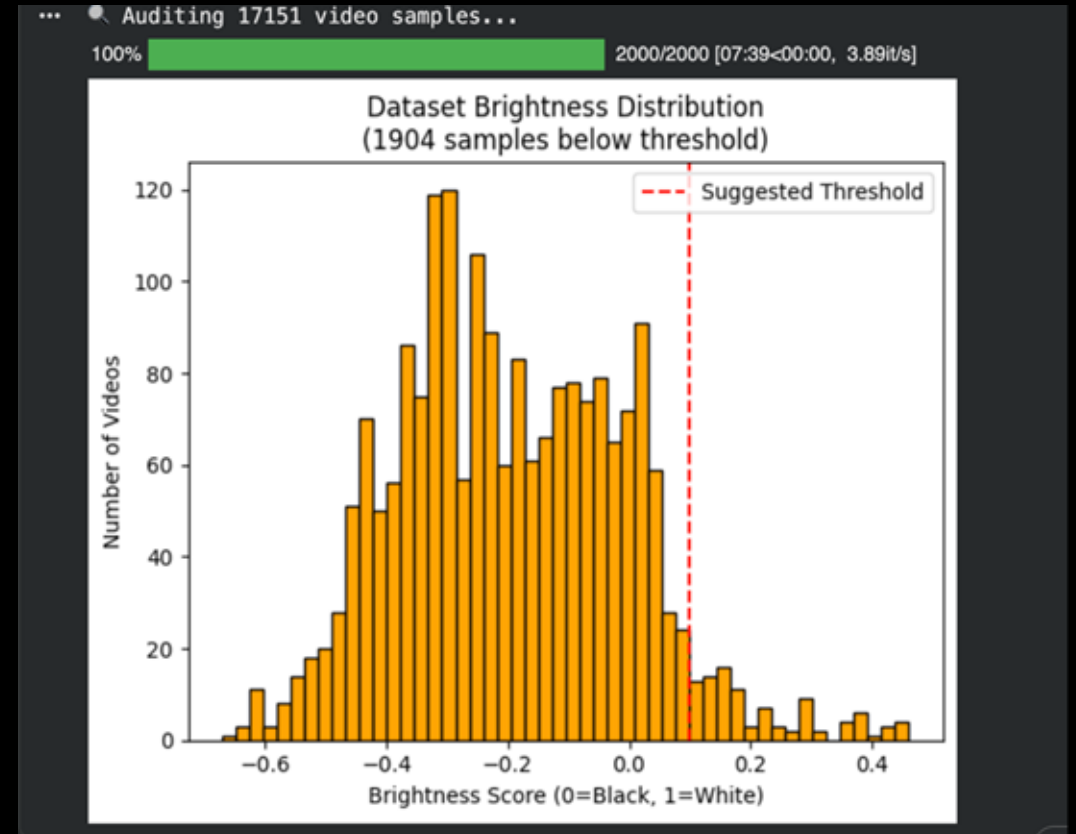
Video Processing

- Extracted **10 frames per video**
- Used **MTCNN** for face detection and cropping
- Focus on **facial regions only**
- Removes **background noise & irrelevant features**
- Improves model focus on **lip movement & expressions**
- This also reduces computational cost while preserving the important features



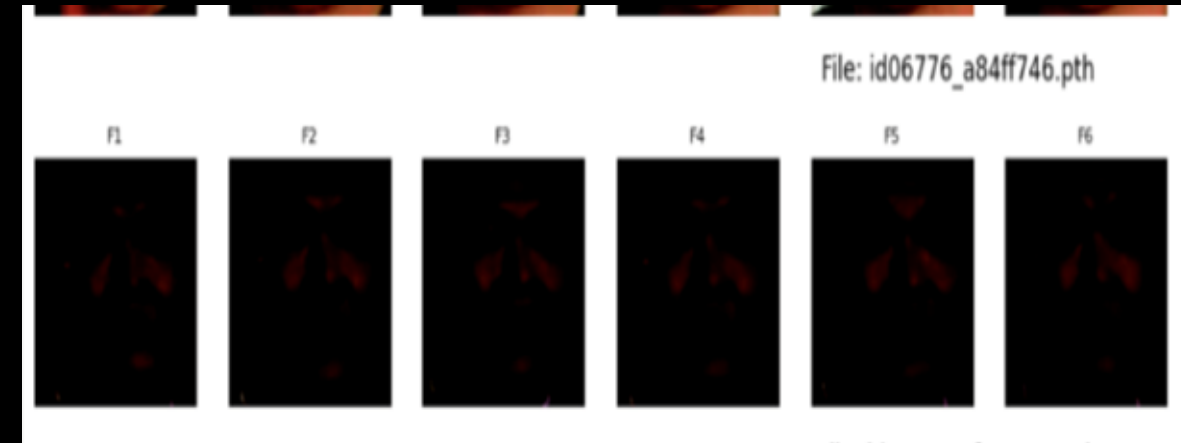
Video Processing - Cause for Concern

- During preprocessing, we detected **brightness imbalance in dataset**
- ~80% of frames were **below brightness threshold**
- Risk: model may learn **lighting instead of features**
- Could lead to **biased or unreliable predictions**
- Required further investigation



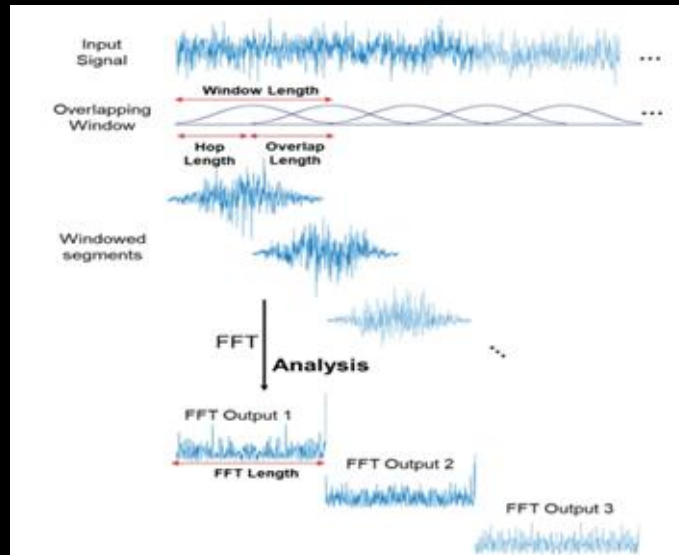
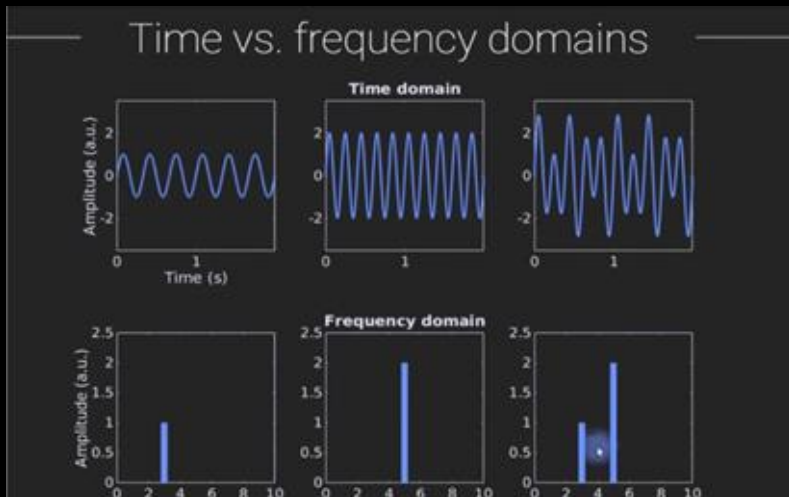
Video Processing - Mistake Correction

- Low brightness Issue actually caused by **MTCNN normalization**
- Pixel values scaled from $[0, 255] \rightarrow [-1, 1]$
- Images appeared **visually darker but were correctly processed**
- Model receives **normalized tensors (not raw images)**
- Prevents **vanishing/exploding gradients**
- Ensures **stable and efficient training**



Audio Processing: Fourier Transforms & STFT

- Raw audio is a **time-domain signal** (amplitude over time)
- Converted to **frequency domain** using STFT
- Splits audio into **small overlapping windows**
- Captures **frequency + intensity over time**
- Produces a **time-frequency representation**
- Foundation for creating **spectrograms**



	0-20 ms	10-30 ms	20-40 ms	30-40 ms
5 Hz	38.28	32.87	36.39	31.13
10 Hz	20.83	28.94	40.91	42.01
20 Hz	40.26	0.00	40.42	40.17
30 Hz	0.00	38.89	37.15	40.91
40 Hz	33.98	34.32	34.81	37.95
50 Hz	42.14	23.52	42.14	42.01
60 Hz	37.27	31.60	34.15	33.06
70 Hz	24.08	31.60	40.09	41.66
80 Hz	0.00	20.00	6.02	19.08

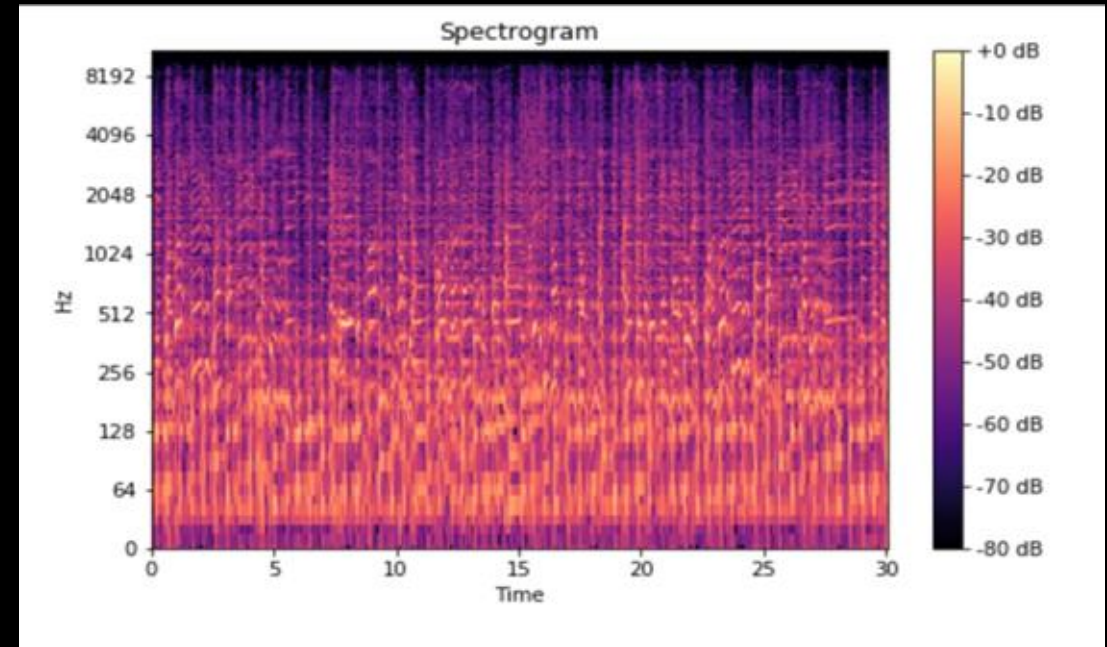
Audio Processing - Mel Spectrogram

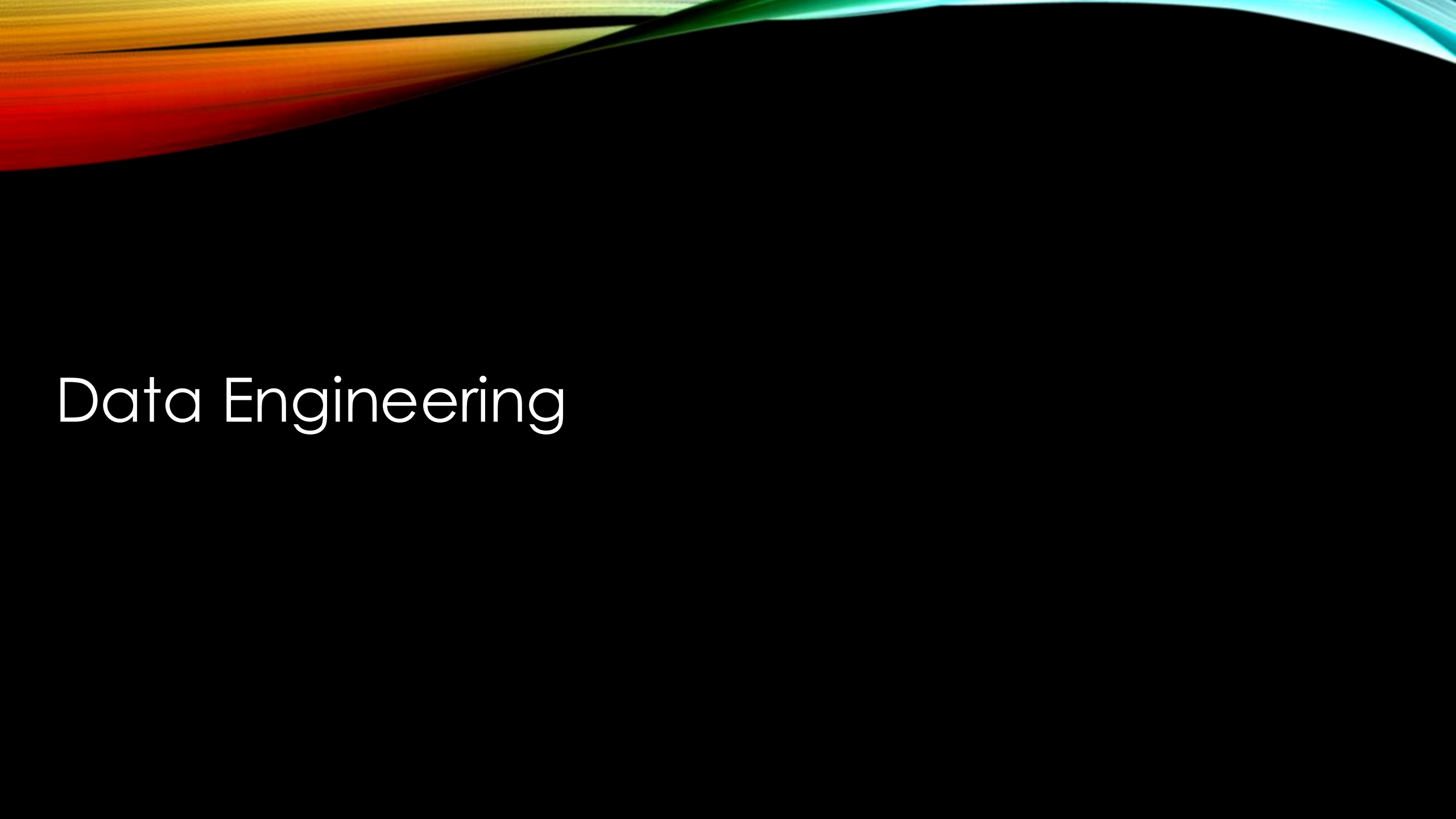
- STFT output converted into a **Mel Spectrogram**
- Applies **Mel scale** to match human hearing
- Converts values to **log scale (decibels)**
- Produces an **image-like representation of audio**
- Used as input for **CNN-based models**

$$\text{MelSpectrogram}(t, m) = 10 \cdot \log_{10}(\sum (\text{Filter}_m \cdot |A|^2))$$

If you were writing this in Python (using `librosa`), the function `librosa.feature.melspectrogram` does exactly this:

1. Takes the **STFT** $|A|$.
2. Squares it to get **Power** $|A|^2$.
3. Multiplies by the **Mel Filterbank** matrix.
4. Applies the **Log** (using `power_to_db`).





Data Engineering

Meta-Data Processing

Synchronization

- Ensured **perfect alignment of audio–video pairs**
- Used **hashing** for deterministic matching
- Guarantees consistency across **multiple processing runs**
- Prevents **misaligned training samples**

Data Leakage Prevention

- Applied **Speaker-Level Split (80/20)**
- Ensures **no identity overlap** between train & test sets
- Prevents model from **memorizing specific faces/voices**
- Improves **true generalization performance**

We made sure our data is clean, aligned, and unbiased

Without this step, the model could memorize identities instead of actually learning deep fake patterns.

Data Engineering - Dataset Preparation

Data Organization

- Data stored in **nested speaker-level folders**
- Each folder contains **multiple audio samples per speaker**
- Required restructuring for **model-ready input pipeline**

Data Extraction & Sampling

- Used `.rglob()` (`pathlib`) to traverse nested directories
- Extracted `.wav` / `.flac` audio files **efficiently**
- Applied **controlled sampling per speaker**
- Maintains **balanced and diverse dataset**

We turned messy real-world data into a clean ML-ready pipeline!

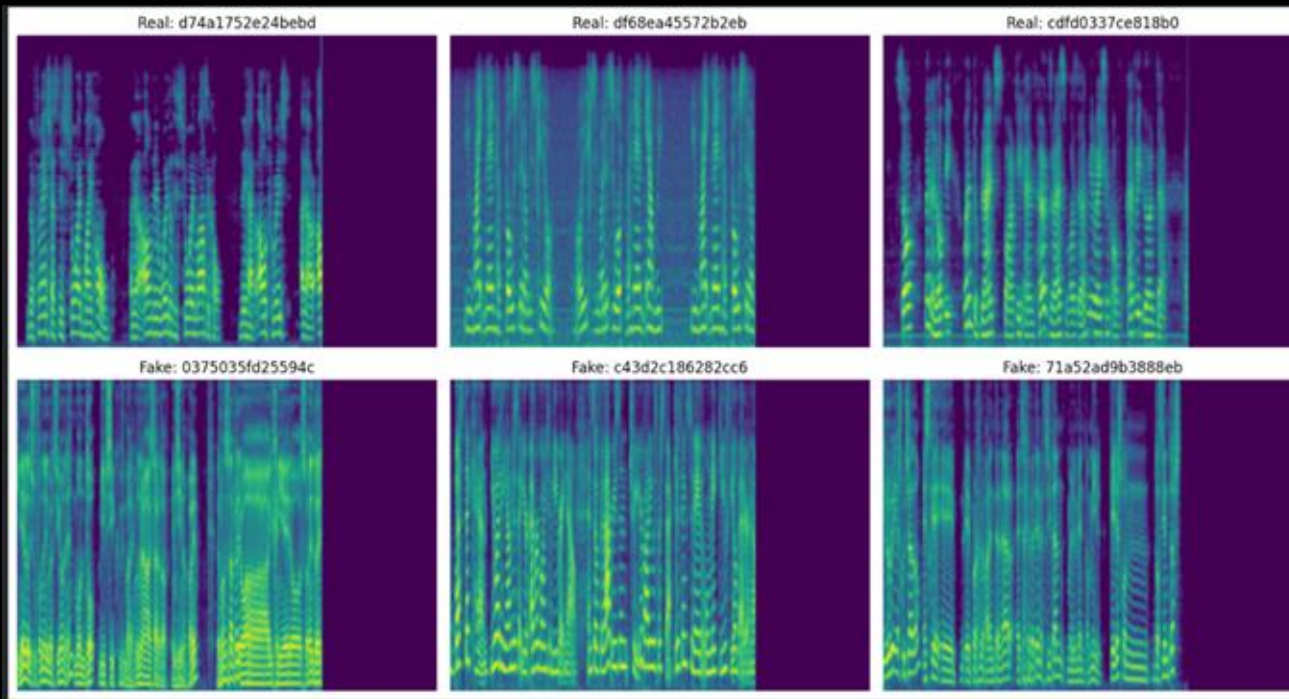
Nested Folders → Extract Files → Speaker Split → Final Dataset

```
# 2. Speaker-Level Split for REAL Data
all_speakers = [d for d in REAL_SRC.iterdir() if d.is_dir()]
random.shuffle(all_speakers)

split_idx = int(len(all_speakers) * TRAIN_SPLIT)
train_speakers = all_speakers[:split_idx]
validation_speakers = all_speakers[split_idx:]
```

```
def process_audio_group(speaker_list, phase):
    print(f"--- Processing {phase.upper()} Real Speakers ---")
    for speaker in speaker_list:
        all_files = list(speaker.rglob('*.flac'))
        # Adjust sampling logic: take up to 20 files per speaker
        selected = random.sample(all_files, min(len(all_files), 20))
```

Data Engineering - torchaudio.transforms



Imagine a photo that is too small for a frame:

No change: You put the photo in the middle and leave a thick black border around it.

Alternative/Reflect: You place mirrors around the photo so the colors and shapes of the image bleed out to the edges of the frame.

This forces the CNN to look at finer details rather than sharp edges in fake audio

Data Engineering - librosa(mode "reflect")

Problem: Edge Effects in Audio

- Audio segments often require **fixed input size**
- Padding with zeros creates **artificial sharp edges**
- Model may learn **padding artifacts** instead of real features

Solution: Reflect Padding

- Used **librosa (mode = "reflect")**
- Extends signal by **mirroring existing patterns**
- Preserves **natural continuity** in audio signals
- Avoids introducing **unnatural boundaries**

Impact

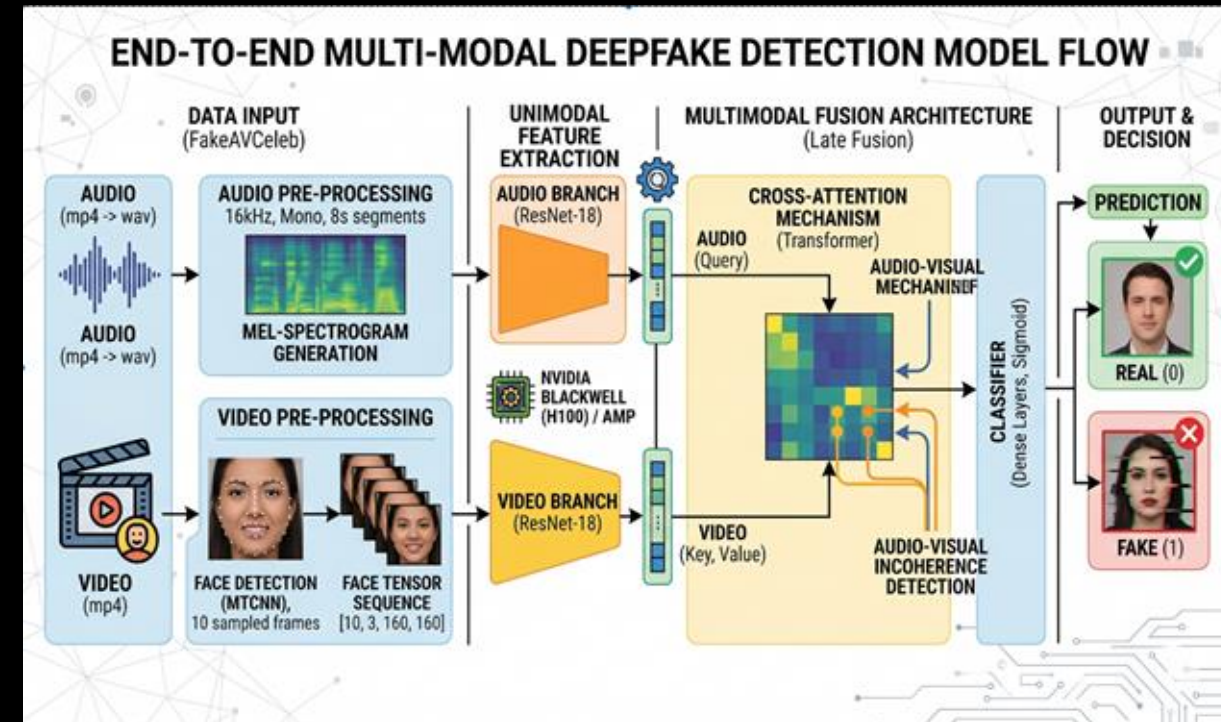
- Produces **smoother spectrograms**
- Helps CNN focus on **meaningful patterns**
- Improves **feature quality** and model robustness

The background of the slide is a solid black field. At the top, there is a decorative horizontal band with a wavy, fluid appearance. This band features a color gradient: on the left, it transitions from yellow to orange; on the right, it transitions from green to cyan. The colors blend into each other, creating a sense of motion and depth.

Model Selection

Model Selection

- Multi-modal architecture combining **audio + video**
- Feature extraction using **ResNet-18 (audio & video branches)**
- Fusion using **Cross-Attention mechanism**
- Learns **audio-visual relationships**
- Detects **incoherence between speech and facial movement**
- Final output: **Real vs Fake classification**



Model Selection

1) Architecture & Feature Extraction

- Dual-branch architecture for **audio and video processing**
- **ResNet-18 backbones** used for both modalities
- Leverages **transfer learning with pretrained weights**
- Efficient balance between **performance and computational cost**

2) Fusion Strategy

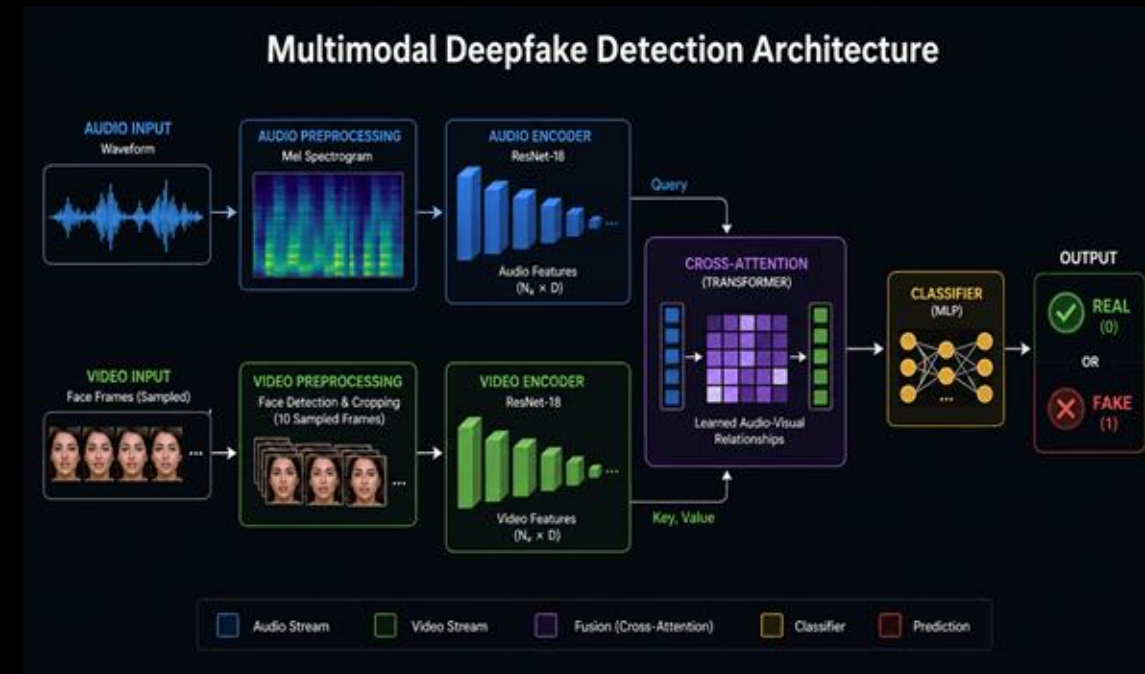
- Uses **Cross-Attention (Transformer-based)** for late fusion
- Audio features act as **query**, video as **key/value**
- Learns **temporal alignment between speech and lip movement**
- Captures **audio-visual dependencies** rather than treating inputs independently

3) Loss Function & Optimization

- **BCEWithLogitsLoss** for stable binary classification
- Combines sigmoid activation + loss for **numerical stability**
- **AdamW optimizer** improves generalization via weight decay
- Handles **noisy gradients common in deep fake data**

4) Training Efficiency

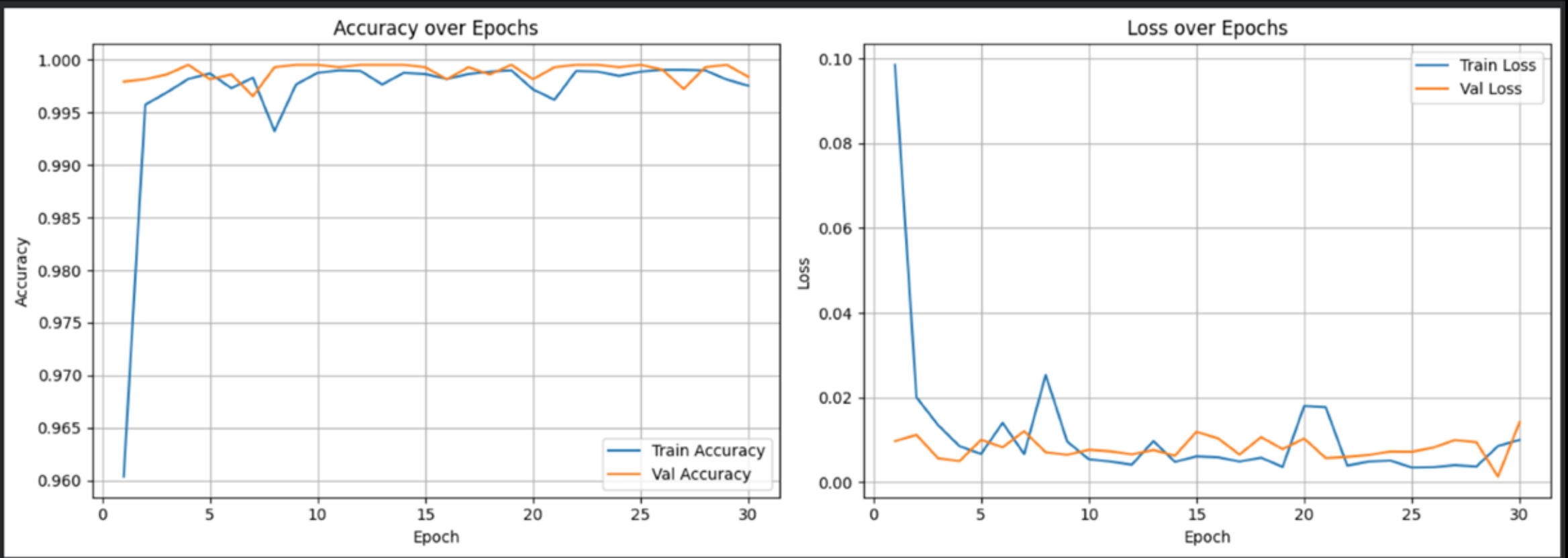
- **Mixed Precision Training (AMP)** reduces memory usage
- Enables faster training on **GPU (G4 / high-performance hardware)**
- Improves **throughput** without sacrificing on **accuracy**.



The background of the slide is a solid black field. At the top, there is a decorative horizontal band with a wavy, fluid appearance. This band features a color gradient: on the left, it transitions from a bright yellow to a deep red; on the right, it transitions from a dark green to a bright cyan. The colors blend into each other, creating a sense of motion and depth.

Conclusion/Discussion

Results from Training & Validation



Results from Training & Validation

Model Performance

- Achieved **~99% accuracy** on both training and validation
- Rapid convergence within **first few epochs**
- Maintained **consistently high performance**

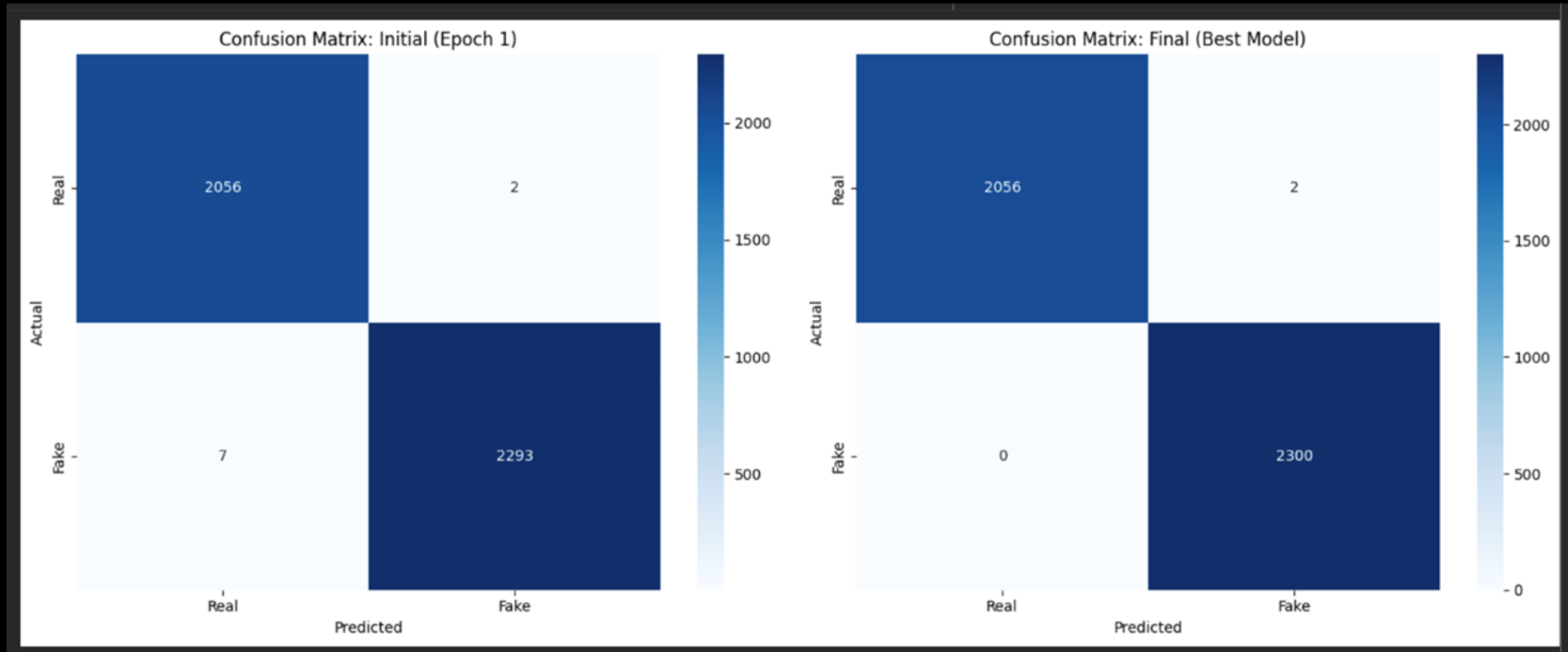
Loss Behavior

- Training loss **decreases quickly and stabilizes**
- Validation loss remains **low and consistent**
- Indicates **stable and effective learning**

Generalization

- Minimal gap between **train and validation curves**
- Suggests **low overfitting**
- Model generalizes well to **unseen data**

Confusion Matrix on Validation Set



Our Findings

Performance Summary

- Near-perfect classification on **validation set**
- Very high **True Positives (Fake detected correctly)**
- Very high **True Negatives (Real detected correctly)**

Error Analysis

- Extremely low **False Positives (Real → Fake)**
- Almost zero **False Negatives (Fake → Real)**
- Indicates strong ability to **detect deepfakes accurately**

Model Improvement

- Clear improvement from **initial → final model**
- Reduction in misclassifications over training
- Shows **effective learning and convergence**

Test - Results in the “Wild”



```
=====
VIDEO FILENAME      | PREDICTION | CONFIDENCE
=====
Morgan_Freeman_F.mp4 | ✓ REAL    | 0.0%
Mark_Zuckerberg._Fmp4.mp4 | ✓ REAL    | 0.2%
Michael_Jackson_F.mp4 | ✓ REAL    | 0.0%
Obama_F.mp4         | ✓ REAL    | 1.1%
Neil_Degrasse.mp4   | ✓ REAL    | 0.8%
```


Discussion

- Deep Fakes are increasingly realistic and difficult to detect
- Unimodal models (audio or video) could have an advantages but we didn't have time to explore and compare unimodal model performances
- As a researcher evaluating 10000+ media files we just want to assign probabilities to potential fakes and conduct further research down the line



Thank You!